

KHOA CÔNG NGHỆ THÔNG TIN

XỬ LÝ ẢNH & THỊ GIÁC MÁY TÍNH

**CHƯƠNG 6: HỌC SÂU VÀ THỊ GIÁC MÁY
TÍNH**

Giảng viên: Nguyễn Phần Lữa

GIỚI THIỆU VỀ HỌC SÂU VÀ THỊ GIÁC MÁY TÍNH

Khóa học COMP8536 tập trung vào việc xây dựng và ứng dụng các mô hình học sâu (Deep Learning) trong lĩnh vực thị giác máy tính (Computer Vision) và xử lý ngôn ngữ tự nhiên.

- **Trí tuệ nhân tạo (AI) → Học máy (ML) → Học sâu (DL):** Học sâu là một tập con của học máy, và học máy là tập con của AI
- Trong khóa học này, chúng ta sẽ nghiên cứu các kiến trúc mạng nơ-ron hiện đại để xử lý:
 - Dữ liệu có kích thước cố định: vector đặc trưng n chiều, ảnh $H \times W$ pixel
 - Dữ liệu tuần tự: chuỗi văn bản, video, chuỗi thời gian
- Các bài toán điển hình: phân loại ảnh, phát hiện đối tượng, mô tả ảnh, trả lời câu hỏi về ảnh, dịch máy, điều hướng bằng thị giác và ngôn ngữ

THỰC HÀNH

Cài đặt một mạng nơ-ron đơn giản với PyTorch để phân loại bộ dữ liệu Iris.

```
import torch
import torch.nn as nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Tải dữ liệu
iris = load_iris()
X = iris.data
y = iris.target

# Chuẩn hóa dữ liệu
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Chia dữ liệu
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Chuyển sang tensor
X_train_t = torch.FloatTensor(X_train)
y_train_t = torch.LongTensor(y_train)
X_test_t = torch.FloatTensor(X_test)
y_test_t = torch.LongTensor(y_test)

# Định nghĩa mô hình
class IrisClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim)
        )

    def forward(self, x):
        return self.network(x)

model = IrisClassifier(4, 16, 3)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# Huấn luyện
for epoch in range(100):
    optimizer.zero_grad()
    outputs = model(X_train_t)
    loss = criterion(outputs, y_train_t)
    loss.backward()
    optimizer.step()

# Đánh giá
with torch.no_grad():
    outputs = model(X_test_t)
    _, predicted = torch.max(outputs, 1)
    accuracy = (predicted == y_test_t).float().mean()
    print(f"Độ chính xác: {accuracy:.4f}")
```

CÁC DẠNG BÀI TOÁN VỚI DỮ LIỆU TUẦN TỰ

Cho đến nay, chúng ta đã làm việc với các đối tượng có kích thước cố định. Tuy nhiên, xử lý dữ liệu tuần tự (sequential data) cũng vô cùng quan trọng. Các bài toán này được phân loại dựa trên số lượng phần tử trong chuỗi đầu vào và đầu ra:

- **One-to-Many (Một-nhiều):** Một đầu vào → chuỗi đầu ra
 - Ví dụ: Mô tả ảnh (image captioning) — một ảnh sinh ra một câu mô tả
- **Many-to-One (Nhiều-một):** Chuỗi đầu vào → một đầu ra
 - Ví dụ: Phân loại video, phân tích cảm xúc từ văn bản
- **Many-to-Many (Nhiều-nhiều):** Chuỗi đầu vào → chuỗi đầu ra
 - Ví dụ: Dịch máy (English → French), chú thích video theo thời gian thực

Các ứng dụng tiêu biểu:

1. **Image Captioning:** Sinh mô tả tự nhiên cho ảnh
2. **Visual Question Answering (VQA):** Trả lời câu hỏi về nội dung ảnh
3. **Video Classification:** Phân loại nội dung video
4. **Natural Language Translation:** Dịch văn bản giữa các ngôn ngữ
5. **Vision-and-Language Navigation (VLN):** Điều hướng robot bằng chỉ dẫn ngôn ngữ và tín hiệu thị giác

PHÂN LOẠI NHỊ PHÂN (BINARY CLASSIFICATION)

Bài toán phân loại nhị phân là nền tảng của nhiều ứng dụng trong thị giác máy tính, chẳng hạn như phát hiện khuôn mặt (faces vs. not faces).

- **Dữ liệu huấn luyện:** $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ với $\mathbf{x}^{(i)} \in \mathbb{R}^n$ và $y^{(i)} \in \{0, 1\}$
 - Nhãn 1: mẫu dương (positive), ví dụ: khuôn mặt
 - Nhãn 0: mẫu âm (negative), ví dụ: không phải khuôn mặt
- **Bài toán hồi quy (Regression):** Học hàm mật độ $P(y|\mathbf{x})$ — ước lượng xác suất mẫu thuộc lớp dương

- **Hàm mất mát Cross-Entropy:**

$$\ell(\theta; \mathbf{x}, y) = -\log[\text{softmax}(f(\mathbf{x}; \theta))]_y$$

- **Cập nhật tham số:**

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

THỰC HÀNH

Bài tập thực hành: Xây dựng mô hình phân loại nhị phân cho bài toán phát hiện spam.

```
import torch
import torch.nn as nn

class BinaryClassifier(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.linear = nn.Linear(input_dim, 1)

    def forward(self, x):
        return torch.sigmoid(self.linear(x))

# Ví dụ với dữ liệu giả lập
torch.manual_seed(42)
X = torch.randn(200, 10)
y = (X[:, 0] + X[:, 1] > 0).float().unsqueeze(1)

model = BinaryClassifier(10)
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    optimizer.zero_grad()
    preds = model(X)
    loss = criterion(preds, y)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 20 == 0:
        with torch.no_grad():
            predicted = (preds > 0.5).float()
            acc = (predicted == y).float().mean()
            print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}, Accuracy: {acc:.4f}")
```

PHÂN LOẠI ĐA LỚP (MULTI-CLASS CLASSIFICATION)

Mở rộng từ phân loại nhị phân, bài toán phân loại đa lớp gán **đúng một nhãn** trong tập K lớp cho mỗi mẫu dữ liệu.

- **Dữ liệu huấn luyện:** $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ với $y^{(i)} \in \{1, \dots, K\}$
- **Ví dụ điển hình:** Phân loại ảnh trên bộ dữ liệu ImageNet (1000 lớp)
- **Mô hình:** Sử dụng hàm softmax ở lớp đầu ra để chuyển đổi logits thành phân phối xác suất:

$$P(y = k | \mathbf{x}) = \frac{e^{f_k(\mathbf{x})}}{\sum_{j=1}^K e^{f_j(\mathbf{x})}}$$

- **Hàm mất mát:** Cross-Entropy Loss

$$L(\theta) = -\frac{1}{N} \sum_{i=1}^N \log P(y^{(i)} | \mathbf{x}^{(i)}; \theta)$$

THỰC HÀNH

Bài tập thực hành: Phân loại ảnh CIFAR-10 (10 lớp) với CNN đơn giản.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x))) # 32x32 -> 16x16
        x = self.pool(F.relu(self.conv2(x))) # 16x16 -> 8x8
        x = x.view(-1, 64 * 8 * 8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN(num_classes=10)
print(model)
# Sử dụng CrossEntropyLoss (đã tích hợp softmax)
criterion = nn.CrossEntropyLoss()
```

PHÂN LOẠI ĐA NHÃN (MULTI-LABEL CLASSIFICATION)

Khác với phân loại đa lớp (chỉ chọn 1 nhãn), phân loại đa nhãn cho phép **gán nhiều nhãn đồng thời** cho một mẫu dữ liệu.

- **Ứng dụng:** Phân loại thuộc tính ảnh (ví dụ: một ảnh có thể chứa "bầu trời", "cây cối", "con người" cùng lúc)
- **Dữ liệu huấn luyện:** $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$ với $\mathbf{y}^{(i)} \in \{0, 1\}^K$
- **Mô hình:** Sử dụng K hàm sigmoid độc lập thay vì 1 hàm softmax:

$$P(y_k = 1|\mathbf{x}) = \sigma(f_k(\mathbf{x})) = \frac{1}{1 + e^{-f_k(\mathbf{x})}}$$

- **Hàm mất mát:** Binary Cross-Entropy cho từng nhãn, sau đó lấy tổng/trung bình:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left[y_k^{(i)} \log \hat{y}_k^{(i)} + (1 - y_k^{(i)}) \log(1 - \hat{y}_k^{(i)}) \right]$$

THỰC HÀNH

Bài tập thực hành: Xây dựng mô hình phân loại đa nhãn.

```
import torch
import torch.nn as nn

class MultiLabelClassifier(nn.Module):
    def __init__(self, input_dim, num_labels):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_labels)
            # KHÔNG dùng softmax, sẽ dùng sigmoid trong loss
        )

    def forward(self, x):
        return self.network(x)

# Dữ liệu giả lập: 5 nhãn cho mỗi mẫu
num_labels = 5
X = torch.randn(100, 20)
y = torch.randint(0, 2, (100, num_labels)).float()

model = MultiLabelClassifier(20, num_labels)
criterion = nn.BCEWithLogitsLoss() # Kết hợp sigmoid + BCE
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(50):
    optimizer.zero_grad()
    logits = model(X)
    loss = criterion(logits, y)
    loss.backward()
    optimizer.step()
    if (epoch + 1) % 10 == 0:
        with torch.no_grad():
            preds = (torch.sigmoid(logits) > 0.5).float()
            acc = (preds == y).float().mean()
            print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}, Accuracy: {acc:.4f}")
```

CHUẨN HÓA DỮ LIỆU (DATA NORMALIZATION)

Chuẩn hóa dữ liệu là bước tiền xử lý quan trọng, giúp mô hình hội tụ nhanh hơn và ổn định hơn.

Chuẩn hóa toàn phần (Full Whitening):

$$\mathbf{x}^{(i)} \leftarrow \Sigma^{-1}(\mathbf{x}^{(i)} - \mu), \quad i = 1, \dots, N$$

trong đó μ là vector trung bình và Σ là ma trận hiệp phương sai trên tập huấn luyện.

Chuẩn hóa từng chiều (Per-dimension Normalization):

Trên dữ liệu quy mô lớn, việc tính ma trận hiệp phương sai đầy đủ là không khả thi, nên ta chuẩn hóa độc lập từng chiều:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{\sigma_j}, \quad j = 1, \dots, n$$

- μ_j : trung bình của chiều j trên tập huấn luyện
- σ_j : độ lệch chuẩn của chiều j trên tập huấn luyện

Lưu ý quan trọng:

- Chỉ tính μ và σ trên **tập huấn luyện**, sau đó áp dụng cho tập kiểm tra
- Tránh rò rỉ thông tin (data leakage) từ tập kiểm tra vào quá trình huấn luyện

THỰC HÀNH

Bài tập thực hành: So sánh tốc độ hội tụ với và không có chuẩn hóa.

```
import numpy as np
import torch
import torch.nn as nn

# Dữ liệu không chuẩn hóa
X_raw = np.random.randn(500, 5) * np.array([100, 0.01, 50, 1000, 0.001])
y = (X_raw[:, 0] + X_raw[:, 1] > 0).astype(np.float32)

# Chuẩn hóa
mu = X_raw.mean(axis=0)
sigma = X_raw.std(axis=0) + 1e-8
X_norm = (X_raw - mu) / sigma

def train_and_measure(X_data, label, epochs=200):
    X_t = torch.FloatTensor(X_data)
    y_t = torch.FloatTensor(y).unsqueeze(1)
    model = nn.Sequential(nn.Linear(5, 1), nn.Sigmoid())
    criterion = nn.BCELoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
    losses = []
    for epoch in range(epochs):
        optimizer.zero_grad()
        loss = criterion(model(X_t), y_t)
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
    print(f"{label} - Loss cuối: {losses[-1]:.4f}")
    return losses

train_and_measure(X_raw, "Không chuẩn hóa")
train_and_measure(X_norm, "Có chuẩn hóa")
```

ĐƯỜNG CONG HỌC TẬP VÀ ĐỘNG LỰC HỌC TỐI ƯU

Đường cong học tập (learning curves) biểu diễn hiệu suất của mô hình trên tập huấn luyện và tập kiểm tra theo thời gian huấn luyện, giúp chẩn đoán các vấn đề như overfitting và underfitting.

- **Underfitting:** Cả loss huấn luyện và validation đều cao → mô hình quá đơn giản
- **Overfitting:** Loss huấn luyện giảm nhưng loss validation tăng sau một số epoch → mô hình quá phức tạp
- **Hội tụ tốt:** Cả hai loss đều giảm và tiệm cận giá trị thấp

Ví dụ với bộ dữ liệu Iris:

- Trộn ngẫu nhiên 150 mẫu, dùng 75 mẫu đầu để huấn luyện, 75 mẫu sau để validation
- Theo dõi loss/accuracy qua từng epoch để xác định điểm dừng tối ưu

Thuật ngữ quan trọng:

- **Learning dynamics / Optimization dynamics:** Các mẫu và hành vi thể hiện qua đường cong học tập
- Hiểu được các động lực học này giúp cải thiện thiết kế thuật toán học
- Với mô hình lớn (ví dụ ImageNet), sử dụng toàn bộ dữ liệu để tính gradient là không khả thi → cần Mini-batch SGD

THỰC HÀNH

Bài tập thực hành: Vẽ đường cong học tập cho mô hình phân loại.

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Chuẩn bị dữ liệu
iris = load_iris()
X = StandardScaler().fit_transform(iris.data)
X_train, X_val, y_train, y_val = train_test_split(X, iris.target, test_size=0.5, random_state=42)

X_train_t = torch.FloatTensor(X_train)
X_val_t = torch.FloatTensor(X_val)
y_train_t = torch.LongTensor(y_train)
y_val_t = torch.LongTensor(y_val)

model = nn.Sequential(nn.Linear(4, 32), nn.ReLU(), nn.Linear(32, 3))
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

train_losses, val_losses = [], []
for epoch in range(100):
    # Huấn luyện
    model.train()
    optimizer.zero_grad()
    train_loss = criterion(model(X_train_t), y_train_t)
    train_loss.backward()
    optimizer.step()
    train_losses.append(train_loss.item())

    # Validation
    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val_t), y_val_t)
        val_losses.append(val_loss.item())

# Vẽ đường cong
plt.figure(figsize=(8, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Learning Curves')
plt.legend()
plt.grid(True)
plt.show()
```

PHÉP TÍCH CHẬP 2 CHIỀU (2D CONVOLUTION)

Phép tích chập 2D là phép toán cốt lõi trong các mạng CNN, được sử dụng để trích xuất đặc trưng từ ảnh.

- **Khác biệt với tích chập 1D:** Kernel không chỉ trượt theo chiều ngang mà còn trượt theo cả chiều dọc (trên mặt phẳng 2D)
- **Đa kênh (Multi-channel):** Ảnh màu có 3 kênh (R, G, B), mỗi kênh là ma trận $n \times m$. Kernel tích chập phải có cùng số kênh với đầu vào.
- **Output feature map:** Kích thước nhỏ hơn đầu vào (trừ khi dùng padding)

Công thức tính kích thước đầu ra:

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

trong đó: I = kích thước đầu vào, K = kích thước kernel, P = padding, S = stride

Các khái niệm quan trọng:

- **Kernel/Filter:** Ma trận trọng số học được, trượt trên ảnh
- **Stride:** Bước nhảy của kernel
- **Padding:** Thêm viền zero xung quanh ảnh để giữ kích thước
- **Feature Map:** Kết quả đầu ra sau khi áp dụng tích chập

THỰC HÀNH

Bài tập thực hành: Minh họa phép tích chập 2D bằng code.

```
import torch
import torch.nn.functional as F
import numpy as np

# Tạo ảnh đầu vào: batch=1, channels=1, height=5, width=5
image = torch.tensor([
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15],
    [16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25]
], dtype=torch.float32).reshape(1, 1, 5, 5)

# Kernel phát hiện cạnh dọc
kernel = torch.tensor([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=torch.float32).reshape(1, 1, 3, 3)

# Tích chập 2D (không padding, stride=1)
output = F.conv2d(image, kernel, padding=0, stride=1)
print("Ảnh đầu vào (5x5):")
print(image.squeeze())
print("\nKernel (3x3):")
print(kernel.squeeze())
print(f"\nKết quả tích chập ({output.shape[-2]}x{output.shape[-1]}):")
print(output.squeeze())

# Ví dụ với ảnh màu 3 kênh
color_image = torch.randn(1, 3, 32, 32) # batch=1, 3 kênh, 32x32
color_kernel = torch.randn(16, 3, 3, 3) # 16 filter, mỗi filter 3x3x3
output_color = F.conv2d(color_image, color_kernel, padding=1)
print(f"\nĐầu vào: {color_image.shape} → Đầu ra: {output_color.shape}")
```

PHÁT HIỆN ĐỐI TƯỢNG BẰNG CỬA SỐ TRƯỢT (SLIDING WINDOW DETECTION)

Phương pháp cửa sổ trượt là cách tiếp cận cơ bản cho bài toán phát hiện đối tượng.

Quy trình:

1. **Đặt bounding box** với kích thước ban đầu tại vị trí đầu tiên trên ảnh
2. **Trích xuất vùng ảnh** bên trong bounding box và đưa vào bộ phân loại (classifier)
3. **Trượt bounding box** sang vị trí tiếp theo (tương tự trượt filter trong tích chập 2D)
4. **Lặp lại** cho đến khi đã quét hết mọi vị trí trên ảnh
5. **Tăng kích thước bounding box** và lặp lại toàn bộ quy trình để phát hiện đối tượng ở các tỷ lệ khác nhau
6. Tiếp tục cho đến khi đã đánh giá ở tất cả các tỷ lệ quan tâm

Hạn chế:

- Rất tốn kém về mặt tính toán (phải phân loại ở mỗi vị trí $\times$ mỗi tỷ lệ)
- Không hiệu quả với ảnh độ phân giải cao
- Các phương pháp hiện đại hơn: R-CNN, YOLO, SSD

PHÂN VÙNG NGŨ NGHĨA (SEMANTIC SEGMENTATION)

Phân vùng ngữ nghĩa là bài toán gán nhãn lớp cho **từng pixel** trong ảnh.

- **Mô tả:** Tương tự phân loại ảnh nhưng áp dụng cho mọi pixel — trả lời câu hỏi "Pixel này thuộc lớp nào?"
- **Đầu ra:** Bản đồ nhãn (label map) có cùng kích thước với ảnh đầu vào
- **Hạn chế:** Không phân biệt được các **thực thể** (instance) khác nhau trong cùng một lớp
 - Ví dụ: Nếu có 3 người trong ảnh, semantic segmentation chỉ cho biết pixel nào là "người" mà không phân biệt người A, B, C

Instance Segmentation = Object Detection + Semantic Segmentation:

- Vừa xác định bounding box của từng đối tượng
- Vừa tạo mặt nạ pixel cho từng đối tượng riêng biệt
- Trả lời câu hỏi: "Pixel này thuộc thực thể nào?"

KIẾN TRÚC U-NET CHO PHÂN VÙNG ẢNH

U-Net (Ronneberger et al.) là kiến trúc encoder-decoder phổ biến cho các bài toán phân vùng ảnh.

Cấu trúc chính:

- **Encoder (Đường co):** Chuỗi các lớp tích chập + pooling, giảm kích thước không gian, tăng số kênh đặc trưng
- **Decoder (Đường mở):** Chuỗi các lớp upsample + tích chập, khôi phục kích thước không gian
- **Skip Connections:** Kết nối trực tiếp các feature map từ encoder sang decoder cùng cấp, giúp bảo toàn thông tin chi tiết không gian

Ưu điểm:

- Hoạt động tốt với ít dữ liệu huấn luyện
- Skip connections giúp khôi phục chi tiết bị mất trong quá trình downsampling
- Được sử dụng rộng rãi trong phân tích ảnh y tế

THỰC HÀNH

Bài tập thực hành: Cài đặt U-Net đơn giản.

```
import torch
import torch.nn as nn

class DoubleConv(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.conv(x)

class UNet(nn.Module):
    def __init__(self, in_channels=3, num_classes=2):
        super().__init__()
        # Encoder
        self.enc1 = DoubleConv(in_channels, 64)
        self.enc2 = DoubleConv(64, 128)
        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.bottleneck = DoubleConv(128, 256)

        # Decoder
        self.up2 = nn.ConvTranspose2d(256, 128, 2, stride=2)
        self.dec2 = DoubleConv(256, 128) # 128 + 128 (skip)
        self.up1 = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.dec1 = DoubleConv(128, 64) # 64 + 64 (skip)

        self.out_conv = nn.Conv2d(64, num_classes, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        b = self.bottleneck(self.pool(e2))
        d2 = self.dec2(torch.cat([self.up2(b), e2], dim=1))
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))
        return self.out_conv(d1)

model = UNet(in_channels=3, num_classes=2)
x = torch.randn(1, 3, 128, 128)
out = model(x)
print(f"Input: {x.shape} → Output: {out.shape}")
```

MÔ HÌNH SEGMENT ANYTHING (SAM)

Segment Anything Model (SAM) của Meta AI là mô hình nền tảng (foundation model) cho bài toán phân vùng ảnh.

Đặc điểm nổi bật:

- Có khả năng phân vùng **bất kỳ đối tượng nào** trong ảnh khi được cung cấp prompt (điểm, bounding box, hoặc mặt nạ)
- Được huấn luyện trên quy mô cực lớn:
 - 1 tỷ pseudo-ground-truth masks
 - 11 triệu ảnh
- Dữ liệu được khởi tạo từ:
 - 4.3 triệu masks được chú thích bởi con người trên 120k ảnh
 - 5.9 triệu masks tự động sinh trên 180k ảnh (được lọc kỹ để loại bỏ nhãn kém chất lượng)

Kiến trúc:

- Image Encoder: Trích xuất đặc trưng từ ảnh (thường dùng ViT)
- Prompt Encoder: Mã hóa prompt (điểm, box, mask)
- Mask Decoder: Sinh mặt nạ phân vùng từ đặc trưng ảnh và prompt

Ý nghĩa: SAM mở ra hướng tiếp cận "promptable segmentation", tương tự cách các mô hình ngôn ngữ lớn hoạt động với prompt.

CƠ CHẾ ATTENTION (CƠ CHẾ CHÚ Ý)

Cơ chế attention cho phép mô hình tập trung vào các phần quan trọng của đầu vào khi sinh ra từng phần tử đầu ra.

Nguyên lý hoạt động:

- Mỗi token đầu ra y_i "chú ý" (attend) đến các token đầu vào x_j một cách có trọng số
- Trọng số attention phản ánh mức độ ảnh hưởng của x_j lên y_i

Trực quan hóa Attention:

- Hiển thị ma trận attention dưới dạng heatmap
- Các hàng: token đầu ra, các cột: token đầu vào
- Màu sắc thể hiện trọng số attention (sáng = chú ý nhiều)

Self-Attention:

- Query (Q), Key (K), Value (V) đều được tính từ cùng một chuỗi đầu vào
- Công thức: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- Hệ số $\frac{1}{\sqrt{d_k}}$ giúp ổn định gradient

Lưu ý:

- Tham số bias thường không được sử dụng vì Layer Normalization sẽ loại bỏ các offset tuyến tính

THỰC HÀNH

Bài tập thực hành: Cài đặt Self-Attention.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, x, mask=None):
        batch_size, seq_len, _ = x.size()

        Q = self.W_q(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(batch_size, seq_len, self.num_heads, self.d_k).transpose(1, 2)

        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)
        attn_weights = F.softmax(scores, dim=-1)

        context = torch.matmul(attn_weights, V)
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, -1)
        return self.W_o(context), attn_weights

# Test
attn = SelfAttention(d_model=64, num_heads=4)
x = torch.randn(2, 10, 64)
out, weights = attn(x)
print(f"Input: {x.shape}, Output: {out.shape}, Attention weights: {weights.shape}")
```

KIẾN TRÚC TRANSFORMER (1)

Transformer là kiến trúc nền tảng được thiết kế ban đầu cho bài toán dịch máy, nay được ứng dụng rộng rãi trong NLP và thị giác máy tính.

Kiến trúc tổng quan:

- **Encoder Stack:** Xử lý chuỗi đầu vào (ví dụ: câu tiếng Anh)
- **Decoder Stack:** Sinh chuỗi đầu ra (ví dụ: câu tiếng Pháp)
- Mỗi stack gồm nhiều lớp, mỗi lớp có:
 - Multi-Head Self-Attention
 - Feed-Forward Network
 - Layer Normalization
 - Residual Connections

KIẾN TRÚC TRANSFORMER (2)

Layer Normalization:

$$y_i = \gamma \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

- μ, σ : trung bình và độ lệch chuẩn của x_i
- γ, β : tham số học được
- Mở rộng cho tensor feature map: tính trung bình và độ lệch chuẩn trên mọi phần tử trong feature map (xử lý độc lập từng mẫu trong batch)

Đặc điểm quan trọng:

- Decoder có thể nhận chuỗi mục tiêu chưa hoàn chỉnh
- Mục tiêu: dự đoán phân phối xác suất của token tiếp theo
- Masked attention trong decoder ngăn chặn việc "nhìn trước" các token tương lai

TOKENIZATION VÀ WORD EMBEDDINGS

Trước khi đưa văn bản vào mô hình, cần chuyển đổi từ thành các biểu diễn số (token và embedding).

Quy trình:

1. **Tokenization:** Chia văn bản thành các đơn vị nhỏ hơn (word, sub-word, hoặc character)
 - Ví dụ: "unhappiness" → ["un", "happi", "ness"]
 - Phổ biến: BPE (Byte Pair Encoding), WordPiece, SentencePiece
2. **Encoding:** Ánh xạ mỗi token $w_t \in \mathcal{V}$ thành vector $z_t = E(w_t) \in \mathbb{R}^d$
 - $E : \mathcal{V} \rightarrow \mathbb{R}^d$ là hàm encoder (thường là bảng tra cứu - lookup table)
 - $\mathcal{V}$: từ vựng cố định

Chuỗi T từ: $\langle w_t \in \mathcal{V} \mid t = 1, \dots, T \rangle$

→ **Chuỗi T token:** $\langle z_t = E(w_t) \in \mathbb{R}^d \mid t = 1, \dots, T \rangle$

Positional Encoding: Vì Transformer không có tính thứ tự bẩm sinh, cần thêm mã hóa vị trí để mô hình biết được thứ tự của các token.

THỰC HÀNH

Bài tập thực hành: Tokenization và embedding đơn giản.

```
import torch
import torch.nn as nn

# Bảng từ vựng đơn giản
vocab = {"<PAD>": 0, "<UNK>": 1, "xin": 2, "chào": 3, "bạn": 4}
embedding_dim = 8

# Embedding layer
embed = nn.Embedding(num_embeddings=len(vocab), embedding_dim=embedding_dim)

# Tokenize
sentence = "xin chào bạn"
tokens = [vocab.get(w, vocab["<UNK>"]) for w in sentence.split()]
token_tensor = torch.LongTensor([tokens])

# Lookup embedding
embedded = embed(token_tensor)
print(f"Tokens: {tokens}")
print(f"Embedded shape: {embedded.shape}") # (1, 3, 8)
print(f"Embedded:\n{embedded}")
```

VISUAL TRANSFORMER (ViT)

Visual Transformer (ViT) áp dụng kiến trúc Transformer cho bài toán xử lý ảnh.

Thách thức:

- Ảnh đã ở dạng số nhưng không phải chuỗi 1D
- Phương pháp đơn giản: duỗi ảnh thành chuỗi pixel (raster scan), mỗi pixel là một token → **không hiệu quả** vì:
 - Một pixel đơn lẻ chứa rất ít thông tin so với một từ
 - Số lượng pixel quá lớn (ví dụ: ảnh 224×224 có hơn 50,000 pixel)

Giải pháp của ViT:

1. Chia ảnh thành các **patch** (ví dụ: 16×16 pixel)
2. Duỗi mỗi patch thành vector và chiếu tuyến tính thành embedding
3. Thêm **[CLS] token** ở đầu chuỗi (tương tự BERT)
4. Thêm **Positional Encoding** để giữ thông tin vị trí
5. Đưa chuỗi patch embeddings vào Transformer Encoder

Ưu điểm:

- Tận dụng được sức mạnh của Transformer và khả năng scale với dữ liệu lớn
- Attention maps cho phép trực quan hóa vùng ảnh mà mô hình chú ý

HỌC ĐẶC TRƯNG ẢNH TỰ GIÁM SÁT VỚI DINO (1)

DINO (self-Distillation with NO labels) là phương pháp huấn luyện ViT mà không cần nhãn, sử dụng kỹ thuật tự giám sát.

Ý tưởng chính:

- Sử dụng cặp mô hình **Teacher** và **Student** cùng kiến trúc
- Hai mô hình học cách tạo ra biểu diễn giống nhau từ các crop/biến đổi ngẫu nhiên khác nhau của cùng một ảnh

Quy trình DINO:

1. Trích xuất hai crop ngẫu nhiên: $x_1, x_2 = T_1(x), T_2(x)$
2. Cập nhật Student bằng gradient descent:

$$\theta_{t+1}^{(1)} = \theta_t^{(1)} - \eta \frac{d}{d\theta^{(1)}} \mathcal{L}_{ce}(p(x_1; \theta_t^{(1)}), p(x_2; \theta_t^{(2)}))$$

3. Cập nhật Teacher bằng EMA (Exponential Moving Average):

$$\theta_{t+1}^{(2)} = \alpha \theta_t^{(2)} + (1 - \alpha) \theta_{t+1}^{(1)}$$

HỌC ĐẶC TRƯNG ẢNH TỰ GIÁM SÁT VỚI DINO (2)

Ứng dụng sau huấn luyện:

- Token [CLS] cho biểu diễn ảnh tốt → dùng cho image retrieval, video segmentation, zero-shot prediction
- Attention maps của DINO hữu ích cho phân vùng ảnh không giám sát và phát hiện vùng nổi bật

Cải tiến: DINOv2 nâng cao chất lượng đặc trưng học được

THỰC HÀNH

Bài tập thực hành: Mô phỏng EMA update trong DINO.

```
import torch
import copy

# Mô phỏng Teacher-Student với EMA
class SimpleModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = torch.nn.Linear(10, 5)
    def forward(self, x):
        return self.fc(x)

student = SimpleModel()
teacher = copy.deepcopy(student)
alpha = 0.996 # EMA coefficient

# Giả lập một bước cập nhật
optimizer = torch.optim.SGD(student.parameters(), lr=0.01)
x = torch.randn(4, 10)

# Student forward + backward
loss = student(x).sum()
optimizer.zero_grad()
loss.backward()
optimizer.step()

# EMA update cho Teacher
with torch.no_grad():
    for teacher_param, student_param in zip(teacher.parameters(), student.parameters()):
        teacher_param.data = alpha * teacher_param.data + (1 - alpha) * student_param.data

print("Teacher parameters updated via EMA")
```

IMAGE CAPTIONING VÀ VISUAL QUESTION ANSWERING (VQA)

Image Captioning:

- Sinh mô tả tự nhiên cho ảnh
- Kiến trúc: CNN (encoder ảnh) + RNN/Transformer (decoder ngôn ngữ)

Visual Question Answering (VQA):

- Kết hợp ảnh và ngôn ngữ để trả lời câu hỏi về ảnh
- Đầu vào: ảnh + câu hỏi (text)
- Đầu ra: câu trả lời (text hoặc chọn từ tập đáp án)

Kiến trúc VQA điển hình:

1. **Image Encoder:** CNN pre-trained (ResNet, ViT) → biểu diễn ảnh
2. **Question Encoder:** RNN/Transformer → biểu diễn câu hỏi
3. **Fusion:** Kết hợp hai biểu diễn (concatenation, attention, hoặc phép nhân)
4. **Decoder:**
 - Câu trả lời mở: mô hình sinh ngôn ngữ (language decoder)
 - Câu trả lời đóng (multiple-choice): MLP + softmax (quy về bài toán phân loại đa nhãn)

THỰC HÀNH

Bài tập thực hành: Mô hình VQA đơn giản (fusion bằng concatenation).

```
import torch
import torch.nn as nn

class SimpleVQA(nn.Module):
    def __init__(self, img_feat_dim=2048, q_feat_dim=512, num_answers=1000):
        super().__init__()
        self.fusion = nn.Linear(img_feat_dim + q_feat_dim, 1024)
        self.classifier = nn.Sequential(
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(1024, num_answers)
        )

    def forward(self, img_feat, q_feat):
        combined = torch.cat([img_feat, q_feat], dim=-1)
        x = self.fusion(combined)
        return self.classifier(x)

model = SimpleVQA()
img = torch.randn(4, 2048) # batch=4, ResNet features
question = torch.randn(4, 512) # batch=4, question features
answers = model(img, question)
print(f"Answer logits shape: {answers.shape}") # (4, 1000)
```

ĐIỀU HƯỚNG BẰNG THỊ GIÁC VÀ NGÔN NGỮ (VLN)

Vision-and-Language Navigation (VLN) là lĩnh vực nghiên cứu kết hợp thị giác và ngôn ngữ để điều hướng tác nhân (agent) trong môi trường.

Bài toán:

- Agent nhận chỉ dẫn bằng ngôn ngữ tự nhiên (ví dụ: "Rời phòng ăn qua cửa bên cạnh bàn, đi đến bếp và dừng trước tủ lạnh")
- Agent phải sử dụng tín hiệu thị giác để thực hiện chỉ dẫn

Phát triển của VLN:

1. **Giai đoạn đầu:** Môi trường ảo với tập vị trí rời rạc, agent chỉ có thể di chuyển giữa các nút đã định nghĩa
2. **Mở rộng:**
 - Khả năng khám phá trước và lập bản đồ môi trường
 - Hoạt động trong môi trường liên tục (continuous)
 - Bổ sung hành động tại đích (ví dụ: "mang cho tôi cái thìa")

ĐIỀU HƯỚNG BẰNG THỊ GIÁC VÀ NGÔN NGỮ (VLN)

Các hướng nghiên cứu hiện tại:

- Embodied AI: Agent tương tác vật lý với môi trường
- Zero-shot navigation: Điều hướng trong môi trường chưa từng thấy
- Multi-modal fusion: Kết hợp nhiều nguồn thông tin (ảnh, ngôn ngữ, bản đồ, cảm biến)

NHÚNG CHUNG THỊ GIÁC VÀ NGÔN NGỮ (CLIP, BLIP)

CLIP (Contrastive Language-Image Pre-training) và BLIP là các phương pháp học biểu diễn chung cho ảnh và văn bản.

Phương pháp Contrastive Learning (CLIP):

- **Đầu vào:** Tập huấn luyện gồm các cặp (ảnh, văn bản)
- **Mục tiêu:** Học sao cho:
 - Biểu diễn của ảnh và văn bản mô tả **tương ứng** gần nhau trong không gian nhúng
 - Biểu diễn của ảnh và văn bản **không tương ứng** xa nhau
- **Loss:** Contrastive loss (InfoNCE)

Ứng dụng:

- **Zero-shot classification:** Phân loại ảnh với nhãn chưa từng thấy trong huấn luyện
- **Image-text retrieval:** Tìm kiếm ảnh bằng văn bản và ngược lại

NHÚNG CHUNG THỊ GIÁC VÀ NGÔN NGỮ (CLIP, BLIP)

- **Open-vocabulary classification:** Gán nhãn bất kỳ danh từ nào cho ảnh, kể cả những nhãn không có trong tập huấn luyện
 - Thường được tăng cường với cơ sở tri thức bên ngoài hoặc mô hình vision-language pre-trained lớn

BLIP (Bootstrapping Language-Image Pre-training):

- Cải tiến từ CLIP
- Thống nhất cả hiểu (understanding) và sinh (generation) trong một mô hình

THỰC HÀNH

Bài tập thực hành: Mô phỏng contrastive loss.

```
import torch
import torch.nn.functional as F

def contrastive_loss(image_embeds, text_embeds, temperature=0.07):
    """
    image_embeds: (batch_size, embed_dim)
    text_embeds: (batch_size, embed_dim)
    Các cặp tương ứng nằm cùng chỉ số.
    """
    # Chuẩn hóa
    image_embeds = F.normalize(image_embeds, dim=-1)
    text_embeds = F.normalize(text_embeds, dim=-1)

    # Similarity matrix
    logits = torch.matmul(image_embeds, text_embeds.T) / temperature
    labels = torch.arange(logits.shape[0], device=logits.device)

    # Symmetric cross-entropy loss
    loss_i2t = F.cross_entropy(logits, labels)
    loss_t2i = F.cross_entropy(logits.T, labels)
    return (loss_i2t + loss_t2i) / 2

# Test
batch_size, embed_dim = 8, 256
img_emb = torch.randn(batch_size, embed_dim)
txt_emb = torch.randn(batch_size, embed_dim)
loss = contrastive_loss(img_emb, txt_emb)
print(f"Contrastive Loss: {loss.item():.4f}")
```

TỰ ĐỘNG VI PHÂN VÀ GỠ LỖI (AUTOMATIC DIFFERENTIATION & DEBUGGING)

Tự động vi phân (Autodiff):

- PyTorch và các framework DL hiện đại cung cấp autodiff để tự động tính gradient
- **Lời khuyên:** Luôn sử dụng autodiff thay vì tính gradient thủ công
- Autodiff giúp tránh lỗi tính toán gradient phức tạp

Chiến lược gỡ lỗi hiệu quả:

1. **Vẽ nhiều đồ thị:** Trực quan hóa loss, accuracy, gradient qua các epoch
2. **Kiểm tra đơn vị (Unit tests):**
 - Overfit một batch nhỏ: loss phải giảm về gần 0
 - Kiểm tra gradient bằng finite difference (numerical gradient checking)
3. **Bắt đầu từ mô hình đơn giản:** Nếu không giải quyết được bài toán, thử giải bài toán đơn giản hơn trước
4. **Chia nhỏ vấn đề:** Tách bài toán lớn thành các bài toán con dễ giải hơn
5. **Đọc bài báo và blog:** Tìm ý tưởng từ các nghiên cứu, nhưng hoài nghi với kết quả không thể tái lập
6. **Rubber Duck Debugging:** Giải thích vấn đề cho một đối tượng (thậm chí là chú vịt cao su) — việc diễn đạt vấn đề thường giúp tìm ra giải pháp
7. **Tải và chạy code của người khác:** Nếu bài báo có code đi kèm, thử chạy để xác minh kết quả

THỰC HÀNH

Bài tập thực hành: Numerical gradient checking.

```
import torch

def numerical_gradient_check(model, x, y, epsilon=1e-5):
    """So sánh gradient tính bằng autodiff và finite difference."""
    loss_fn = torch.nn.MSELoss()

    # Gradient từ autodiff
    model.zero_grad()
    pred = model(x)
    loss = loss_fn(pred, y)
    loss.backward()
    auto_grads = {name: param.grad.clone() for name, param in model.named_parameters()}

    # Gradient bằng finite difference
    for name, param in model.named_parameters():
        grad_numerical = torch.zeros_like(param)
        for i in range(param.numel()):
            orig = param.data.view(-1)[i].item()

            param.data.view(-1)[i] = orig + epsilon
            loss_plus = loss_fn(model(x), y).item()

            param.data.view(-1)[i] = orig - epsilon
            loss_minus = loss_fn(model(x), y).item()

            grad_numerical.view(-1)[i] = (loss_plus - loss_minus) / (2 * epsilon)
            param.data.view(-1)[i] = orig

        diff = (auto_grads[name] - grad_numerical).abs().max()
        print(f"{name}: max diff = {diff:.2e}")

# Test
model = torch.nn.Linear(4, 2)
x = torch.randn(3, 4)
y = torch.randn(3, 2)
numerical_gradient_check(model, x, y)
```